



HiveRelay

BLINDSPARK · PEAR ECOSYSTEM CORE

v0.17 → v0.24.1 · what's new

The Custody, Notify & **Peerit** Release Cycle

Since our last announcement at **0.17**, HiveRelay grew from an app-seeding appliance into a **content-blind service network** — relays that can prove what they store, guard secrets no single operator can open, wake sleeping devices, and keep **Peerit** — a peer-to-peer Reddit with no servers — alive 24/7 without ever reading a byte of your data.

8 releases

Blind custody — k-of-n secret dispersal

Proof-of-Storage

Peerit launch services

P2P in the browser

Security audit hardening

Release announcement

HiveRelay · Blindspark appliance · Pear browser ecosystem

July 2026

v0.24.1

01 The short version

Three storylines run through this cycle:

STORYLINE 1

A network you can verify

Relays now **prove** they hold what they claim, discover each other with no central directory, and split secrets so **no single operator** — and no minority of them — can reconstruct your data.

STORYLINE 2

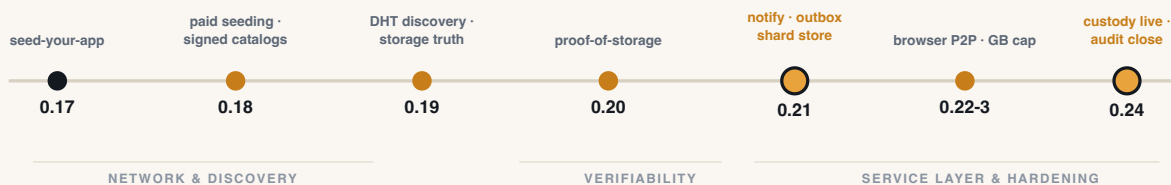
A service layer for apps

Headless, opt-in services apps consume over RPC: encrypted wake-up push, a signed outbox the relay can't read, verifiable randomness, and blind shard storage — the plumbing that makes **Peerit** possible.

STORYLINE 3

Peerit & the browser

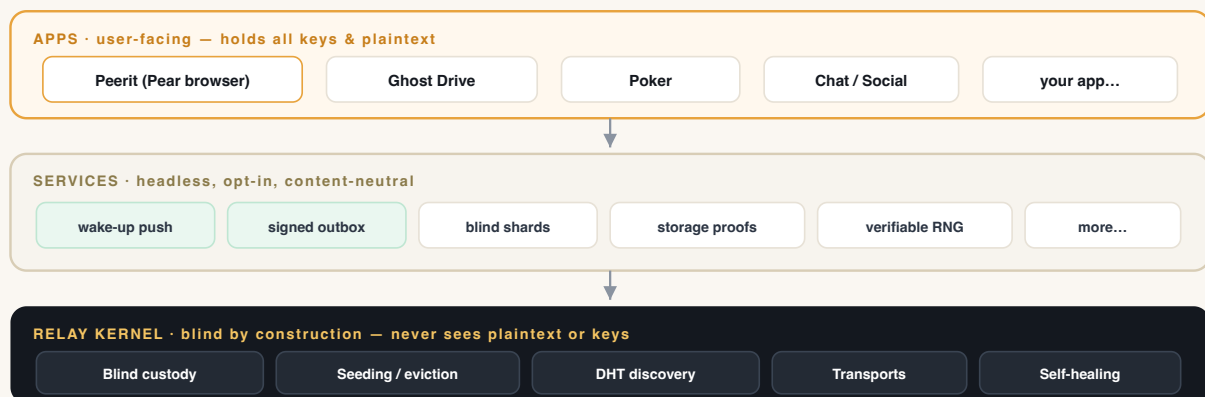
The Pear browser moved to **per-relay subscriptions** with provenance badges, real P2P over WebSocket, and signed catalogs it can verify — no central index, no forced federation.



Eight releases, three phases. The larger amber nodes (0.21, 0.24) carry the Peerit launch.

HOW THE PIECES STACK UP

HiveRelay is a two-layer system: apps consume headless **services** over RPC, and those services sit on a relay **kernel** that is blind to content by construction.



The two-layer architecture. Green services are what Peerit consumes; everything below the line is content-blind.

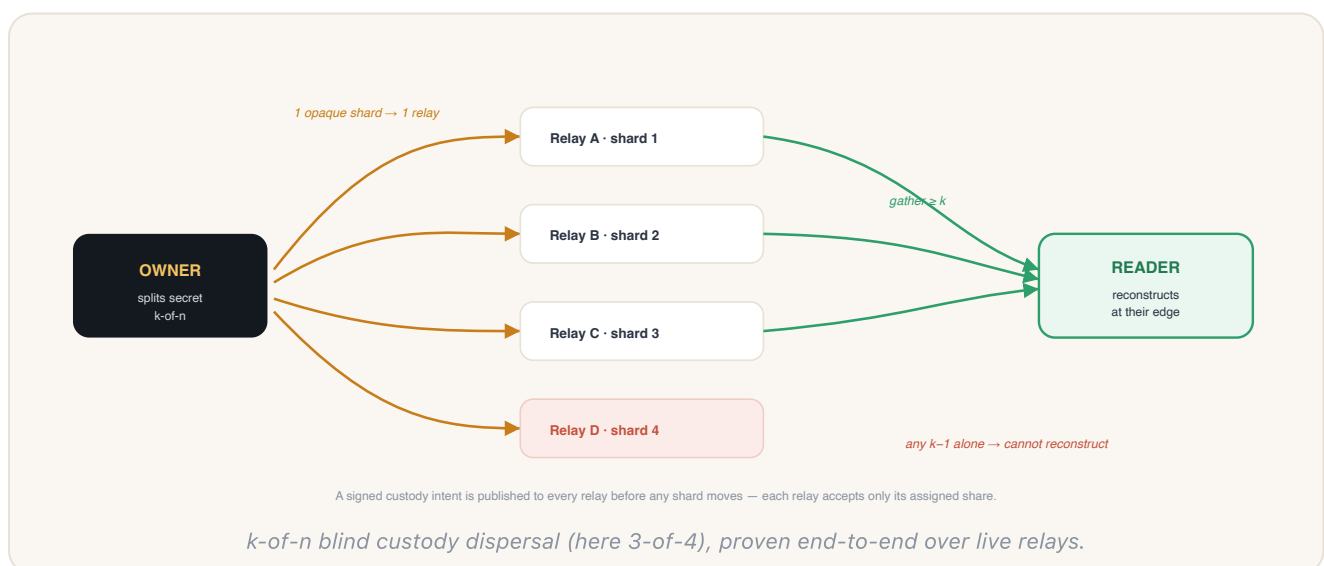
02 Network upgrades

What changed underneath: discovery, verifiability, and honest accounting.

- **Directory-free discovery** 0.19 — a relay publishes a signed self-description straight into the DHT. Anyone who knows a relay's public key can find its current address — **no trusted directory, no central registry**.
- **Proof-of-Storage** 0.20 — you no longer have to trust a relay's word. A client can challenge any relay to **cryptographically prove** it really holds an app: random blocks are sampled and each proof verifies against the app's own signed Merkle root. Forged content is rejected; a privacy gate ensures the mechanism can't be abused to unmask blind drives.
- **Honest storage & serving metrics** 0.19–0.20 — relays now measure their **real on-disk footprint** and real bytes served. This root-caused and fixed the disk-full incidents: the adoption cap now binds on true usage.
- **Give HiveRelay N GB** 0.23 — operators set exactly how much disk the relay may use, live from the dashboard, no restart. The relay shrinks to fit while replication guarantees still protect the network.
- **Paid seeding, zero custody** 0.18 — a relay can charge a publisher to keep their app seeded for a window. Funds settle to the operator's **own Lightning node**; pricing never keys on content; self-hosting stays free.
- **P2P byte-pipe, production-grade** 0.22 — the WebSocket↔DHT transport was hardened for 24/7 operation (liveness supervision, backpressure limits, byte metering). Every bound is content-neutral — the relay pipes encrypted bytes it cannot read, preserving its common-carrier posture.

Blind shard custody — the headline primitive

Landed in 0.21, wired end-to-end and proven in 0.24. The idea: encrypt content with a random key, then **split the key into n shards and disperse them across independent relays**. Each relay holds one opaque, content-addressed shard it cannot interpret. Any k shards let the owner (or their heirs, or their other devices) reconstruct — but **no single relay, and no $k-1$ colluding relays, can ever produce the plaintext**. Every shard placement is authorized by a signed custody intent published *before* the bytes move, so each relay accepts only the exact share it was assigned.

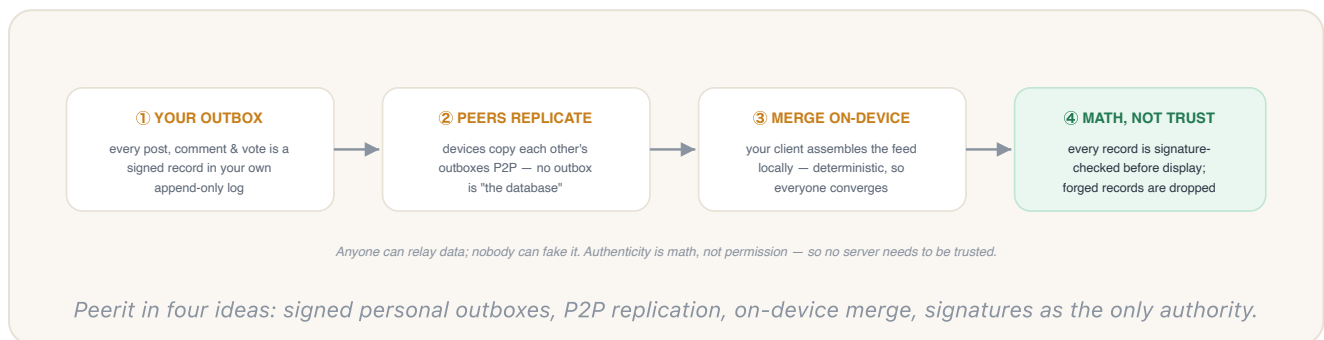


Alongside custody, two new signed logs complete the availability story: independent **witnesses** attest to what relays actually serve, and **repair tickets** drive the network's self-healing loop. *Honest caveat: dispersal has been proven mechanically end-to-end, including against a full four-node test network — the security property (no single operator can reconstruct) still awaits its fleet run across independent operators.*

03 The launch of Peerit

A peer-to-peer Reddit with no servers — and the relay services that keep it alive 24/7.

Peerit is a peer-to-peer Reddit. Communities, posts, threaded comments, votes, moderation, karma — everything you'd expect — with **no servers, no company, and no central database** that can be seized, sold, or censored. The whole app is a folder of HTML and JavaScript; the "backend" is the people using it. It runs fully P2P inside **PearBrowser**, or in any normal browser at **peerit.site** through an untrusted relay.



A pure-P2P forum has one hard problem: **peers sleep**. Phones lock, laptops close, and your posts must stay reachable while you're offline. That's where HiveRelay comes in — Peerit leans on it for exactly two things, and both are built so the relay learns as close to nothing as possible:

ENCRYPTED WAKE-UP PUSH notify

An always-on relay wakes a peer's app through APNs / FCM / WebPush **without ever seeing who is being notified or why**. The device consents to be woken; a peer must hold explicit authorization to wake it; both are revocable. Payloads are opaque ciphertext and wakes are generic — delivery records are signed and redacted, so nothing leaks even to the operator.

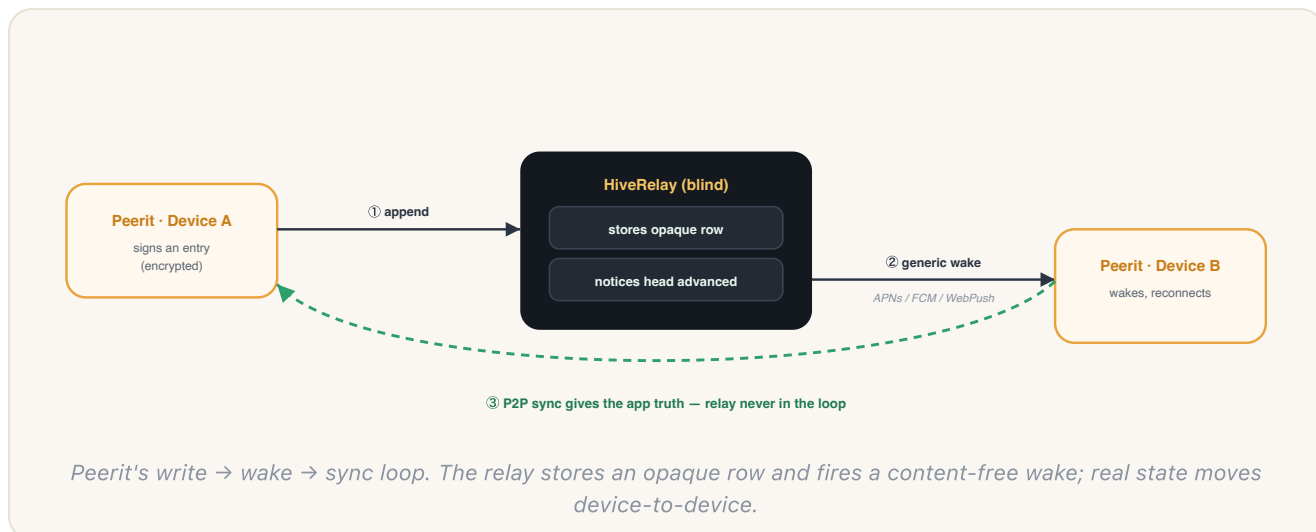
A SIGNED OUTBOX THE RELAY CAN'T READ outboxlog

A per-identity append-only log: only the owner's key can write, every entry is signature-checked, and entry bodies stay **opaque to the relay**. The relay stores and serves rows it cannot read — and on every restart it re-verifies all signatures, so its own disk is never a trust root. Only the writer key is.

THE GOVERNING RULE

"Push wakes the app; P2P sync gives the app truth."

The relay only nudges a device to reconnect. Real state then moves device-to-device — the relay is never a mailbox and never the source of truth.



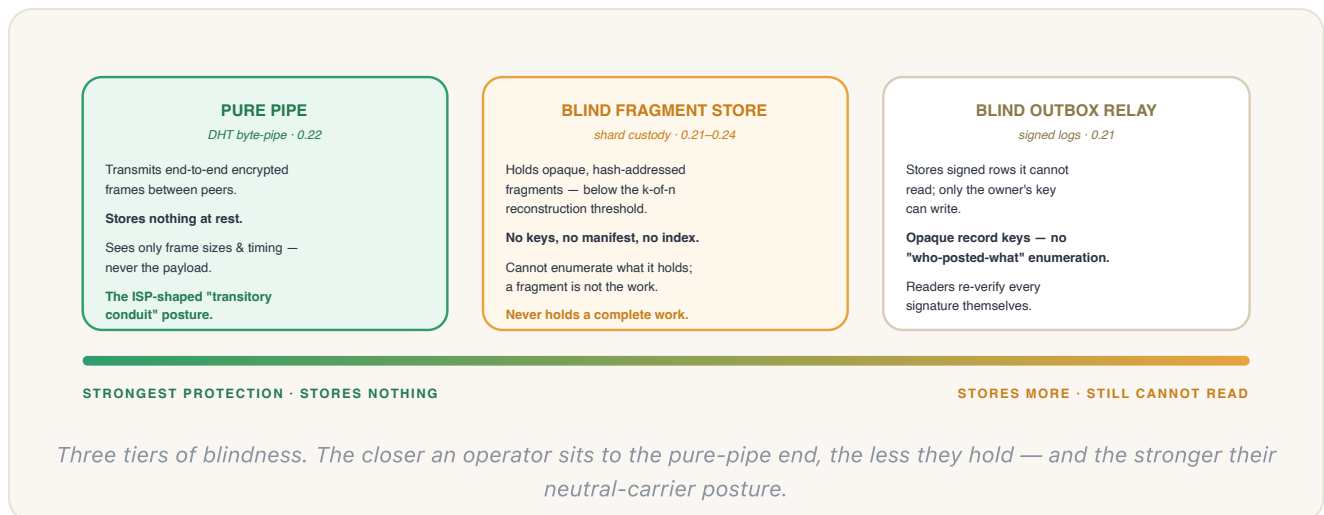
DESIGNED TO STAY HONEST

- **Namespace-bound signatures.** The outbox is app-neutral; every entry is signed under its app's namespace (Peerit signs 'peerit'), and the namespace is bound *inside* the signature — so an entry can never be replayed as another app's.
- **Crash-safe by journal.** Every append is journaled before it's acknowledged; recovery replays the journal, then re-verifies every row's signature.
- **Accountable takedown, blind content 0.24**. Operators get a takedown control for liability parity — it acts on an opaque row ID, never reads content, requires a dedicated admin credential, and simply doesn't exist unless the operator provisions it.
- **One-line provisioning 0.24.1**. The most common launch-day issue — a freshly provisioned box rejecting Peerit's writes because its namespace wasn't registered — is now a single environment variable on the appliance. Fresh boxes work out of the gate.

04 The blind relay model

What "blind" actually means, tier by tier — and the full blind pipe Peerit is moving to.

Every HiveRelay surface is built to the same rule: the relay may **carry, store, and serve** — it may never **read, select, or vouch**. That's not one feature; it's a spectrum of three operating tiers, each blinder than the last:



The full blind pipe — closing both leaks

A conventional relay leaks in **two independent ways**, and each needs its own fix. Peerit's migration composes both into one record:

LEAK 1 — THE SOCIAL GRAPH

Even with encrypted bodies, plaintext record keys let an operator enumerate *who posted where, who voted on what* by simply scanning key prefixes.

The fix: opaque record keys — every key becomes an HMAC'd identifier, so the relay's disk reveals no social structure at all.

LEAK 2 — THE CONTENT KEY

If the decryption key sits next to the ciphertext, the "encryption" is decoration — the operator can read bodies at will.

The fix: the body key is **split k-of-n across the relay cohort** (blind custody, §02). No relay holds it; only a reader gathering k shares can reconstruct.

THE HONEST ONE-LINE CLAIM

"In the ordinary course the operator holds and serves **nothing readable**, must **affirmatively reconstruct** to read — and **cannot enumerate the social graph** by key." That's the whole claim. Peerit is a *public* forum, so content isn't secret — any reader can reconstruct it; the point is the *operator's ordinary course* is blind, not that the data is hidden from the world. Metadata (record counts, timing, sizes) is not hidden either.

Why blindness protects the operator

Intermediary-liability doctrines — safe-harbor loss, vicarious and contributory liability — turn on three prongs. The architecture is deliberately shaped to remove each one, and a plaintiff generally needs **two of them together**:

PRONG

HOW THE DESIGN REMOVES IT

Knowledge

actual or "red-flag" awareness

Blindness — in the ordinary course the operator perceives only opaque bytes. You cannot have red-flag awareness of content you are structurally unable to read.

Attributable benefit

profit tied to the content

Content-neutral pricing — operators are paid (if ever) for **bytes stored, bytes served, and uptime**. Never per-work, never popularity-weighted. Popular contraband and a cat photo are the same random-looking bytes earning the same metered rate.

Right & ability to control

power to supervise or select

No ability to read, rank, or select — plus **blind takedown**: on a valid notice the operator drops the named opaque identifier *without ever reading it*. Blind storage and prompt takedown are not in tension when the target is an ID, not content.

THE ONE SENTENCE THE WHOLE ECONOMICS KEEPS TRUE

"An operator earns the same whether a fragment is lawful or not — because they cannot tell the difference, cannot select what they hold, and never hold a complete work."

"BLIND TAKEDOWN" SOUNDS LIKE A CONTRADICTION. IT ISN'T — THREE REASONS.

- **Immutable means unforgeable, not undeletable.** A takedown is a *serve-time veil*, not a rewrite: the bytes stay on disk, the append-only log and its signed history are untouched, and the named identifier is simply filtered out of every response (which is also why a takedown can be *restored*). Nobody — operator included — can forge, alter, or impersonate; every client still verifies the same signed history. What takedown exercises is the one power a relay always had: choosing whether to serve.
- **Blind ≠ secret — the notice-sender does the reading.** Peerit is a public forum: a complainant reads the content the way any reader does, and the identifiers fall out mechanically — record keys, content-hash IDs, and which relay holds which share (placement is deterministic from the signed roster, and custody receipts are signed records naming each relay's assignment). The complainant maps content → identifiers; the operator only ever matches identifiers. "Blind" was never "nobody knows what's stored" — it's "the operator, in the ordinary course, cannot read or select."
- **Compliance on one box, not global erasure.** Other relays and peers still carry the content — a conduit can only be asked to act on its own service. Censorship-resistance lives in replication across independent operators; takedown-capability lives on each individual box. The two compose: the network stays resilient precisely because no single relay's serving decision is authoritative.

STATED PLAINLY — WHAT THIS IS NOT

- **Not immunity, and not legal advice.** It is a *risk-reduction posture*; jurisdictions vary, and operators should get their own counsel — especially before accepting payment.
- **Not self-executing.** The posture holds only while the operator keeps it: stay blind, stay content-neutral, honor drop-by-ID notices — and never market the relay as "host anything." **Inducement destroys the entire argument regardless of blindness.**
- **Not real until operators are independent.** Blindness holds against independent operators; a fleet secretly run by one entity is one host wearing many hats. That's why the network's next gate is recruiting genuinely arms-length operators — separate entities, hosting, funding, and keys.

05 Pear browser integrations

Discovering apps across a federation of relays you don't have to trust.

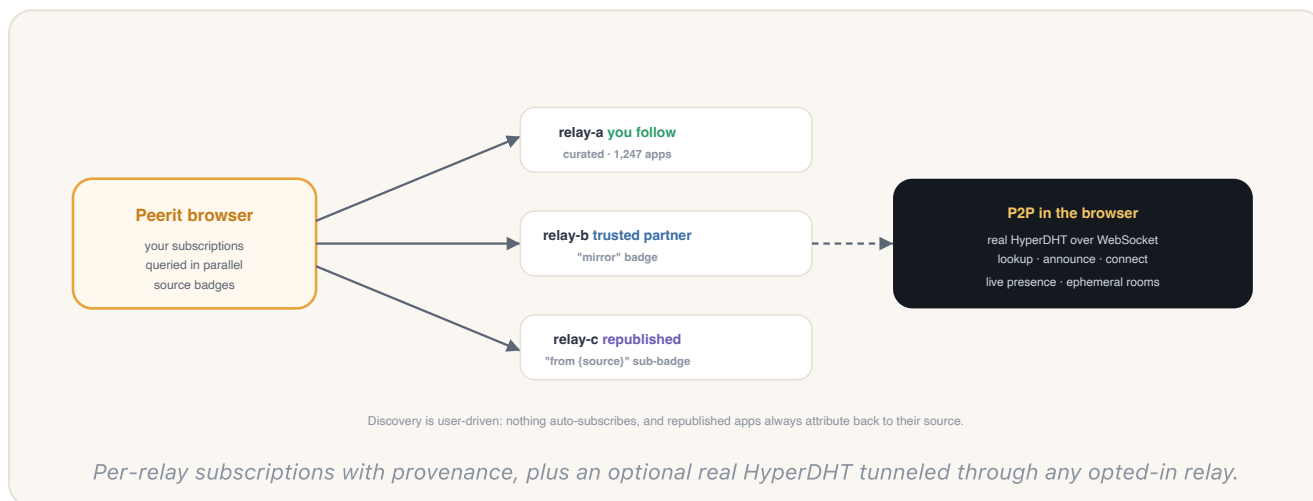
The big shift: away from one global merged catalog, toward **per-relay catalogs you explicitly subscribe to**. Operators curate what their relay carries; the browser queries each relay separately and shows **where every app came from** instead of silently merging. Federation became provenance metadata — not a discovery mechanism.

BEFORE

- One global merged catalog
- Apps propagated between relays without operator consent
- No real P2P from the browser (no UDP)

NOW

- Each relay curates a **local catalog**; you subscribe to the relays you choose
- Every app row carries a **source badge** — mirrors and republishes always attribute back
- A **real HyperDHT in the browser**, tunneled over WebSocket through any opted-in relay



- **Signed catalogs over P2P.** A relay can publish its catalog as a signed, replicable structure the browser verifies cryptographically — preferred over polling HTTP when available.
- **One-tap subscribe.** Encounter a relay through a shared link or QR code and add it with one tap — explicitly, never auto-trusted.
- **Seed & list from the appliance** **0.17**. Operators seed their own app from the dashboard and it auto-lists in their catalog, icon included, ready for the browser to find.
- **Runs inside the browser** **0.20.1**. The client library now loads cleanly in Pear/Bare worker bundles — the browser talks to relays natively, no shims.

06 Security & trust

Before opening the network up, 0.24 closed every HIGH finding from a full audit pass.

CLOSED · HIGH

QUARANTINE NEEDS PROOF

Previously any followed relay could quarantine any app network-wide — an unauthenticated censorship lever. Now quarantine requires **cryptographic evidence of a real fork**, verified against the app's own key, from an allow-listed observer. Forged or unsigned claims do nothing.

CLOSED · HIGH

CUSTODY BYTES CAN'T RACE

Shard storage was hardened against concurrency and disk pressure: bytes are deleted only when every reference agrees, under lock — no sweep can delete live custody data, and retention floors are honored even under pressure.

CLOSED · HIGH

SIGNED SUPPLY CHAIN

Fleet updates verify a trusted signature before any code is checked out (fail-closed); appliance images are pinned by digest and signed with cosign. What runs on your box is what we shipped.

Across the cycle: notification delivery records became caller-authenticated (closing a metadata leak), every HTTP surface gained per-IP rate limiting, proxy deployments got correct per-client limits, and slow readers can no longer balloon relay memory. Releases now auto-sync to the community app store on every version, pinned by image digest, so what you install never lags what we ship.

WHERE THIS GOES NEXT

The custody mechanism is proven end-to-end; the next gate is the **independent-operator fleet run** — dispersal across relays no single party controls. Alongside it: production push-provider adapters for notify, client-only custody dealing, and the browser-side surfaces that put all of this one tap away.

HiveRelay 0.24.1 · July 2026 · Blindspark
appliance · Pear ecosystem



Full technical details, configuration references, and per-release changelogs are in the repository's CHANGELOG.md and docs/. Where a capability is pending an independent-operator gate, this document says so.